RFR27 16-Channel Temperature Monitor New Member Project-10

Rutgers Formula Racing

Maaz Mansuri, Joshua Scallo, Fayella Fenton, Gavin Palacios
November 10, 2026



Contents

1 Revision Log **2**

2 Purpose **2**

3 Parties Involved **2**

4 Hardware Design **3**
4.1 Introduction to Schematic Design . . . 4
4.2 RC Filter . . . 4
4.3 Difference Amplifier . . . 5
4.4 Voltage Divider & Buffer Op-Amp . . . 5
4.5 Active Filtering (2.2 nF Capacitor) . . . 5
4.6 Settling Time Analysis . . . 6
4.7 Verifying 1000Hz Sampling Rate . . . 6
4.8 Buck Converter . . . 6
4.9 Schematic Changes in Implementation . . . 7

5 Firmware Design **7**
5.1 Introduction and Firmware Overview . . . 7
5.2 Pin Mapping and Peripheral Allocation . . . 9
5.3 Overview of functions . . . 9
5.3.1 Channel Addressing: `setMuxChannel` . . . 9
5.3.2 Signal Acquisition: `readChannelVoltage` . . . 10
5.3.3 Mapping our voltage readings into temperature values: `voltagetoTemperature` . . . 10
5.3.4 I2C Implementation: `requestData` . . . 11
5.4 Design Choices . . . 11
5.4.1 Settling Delay Choice (20 μs) . . . 11

6 Bill of Materials **11**

7 Sources **12**

1 Revision Log

Editor	Date	Revision
Maaz Mansuri	Oct. 12 2026	Document Creation
Maaz Mansuri	Oct. 15 2026	Established outline of documentation and table of contents
Fayella Fenton	Oct. 15 2026	Inserted Source Refs
Joshua Scallo	Oct. 17 2026	Added schematic and hardware descriptions
Maaz Mansuri	Oct. 19 2026	Completed firmware Design and added flowcharts for the circuit and the code
Joshua Scallo	Nov. 4 2026	Updated schematic version and completed schematic design justifications.
Joshua Scallo	Nov. 9 2026	Added complete BOM
Maaz Mansuri	Nov. 10 2026	Updated code and revised explanations for firmware design

2 Purpose

Create a 16-channel temperature monitor for the accumulator container and transmit pack thermal data over the I2C bus upon master query.

3 Parties Involved

Name	Team Role	Report Role
Maaz Mansuri	Software	Author
Joshua Scallo	Hardware	Author
Fayella Fenton	Hardware	Author
Vaanika Singla	Electronics Lead	Reviewer
Jeevan Shah	Electronics Lead	Reviewer

4 Hardware Design

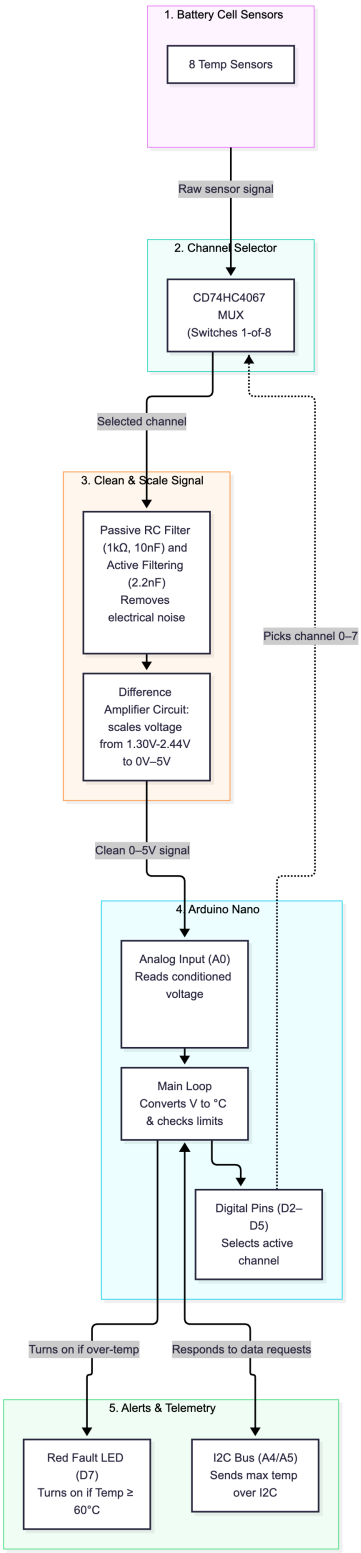


Figure 1: Hardware System Signal-Flow and Architecture Diagram

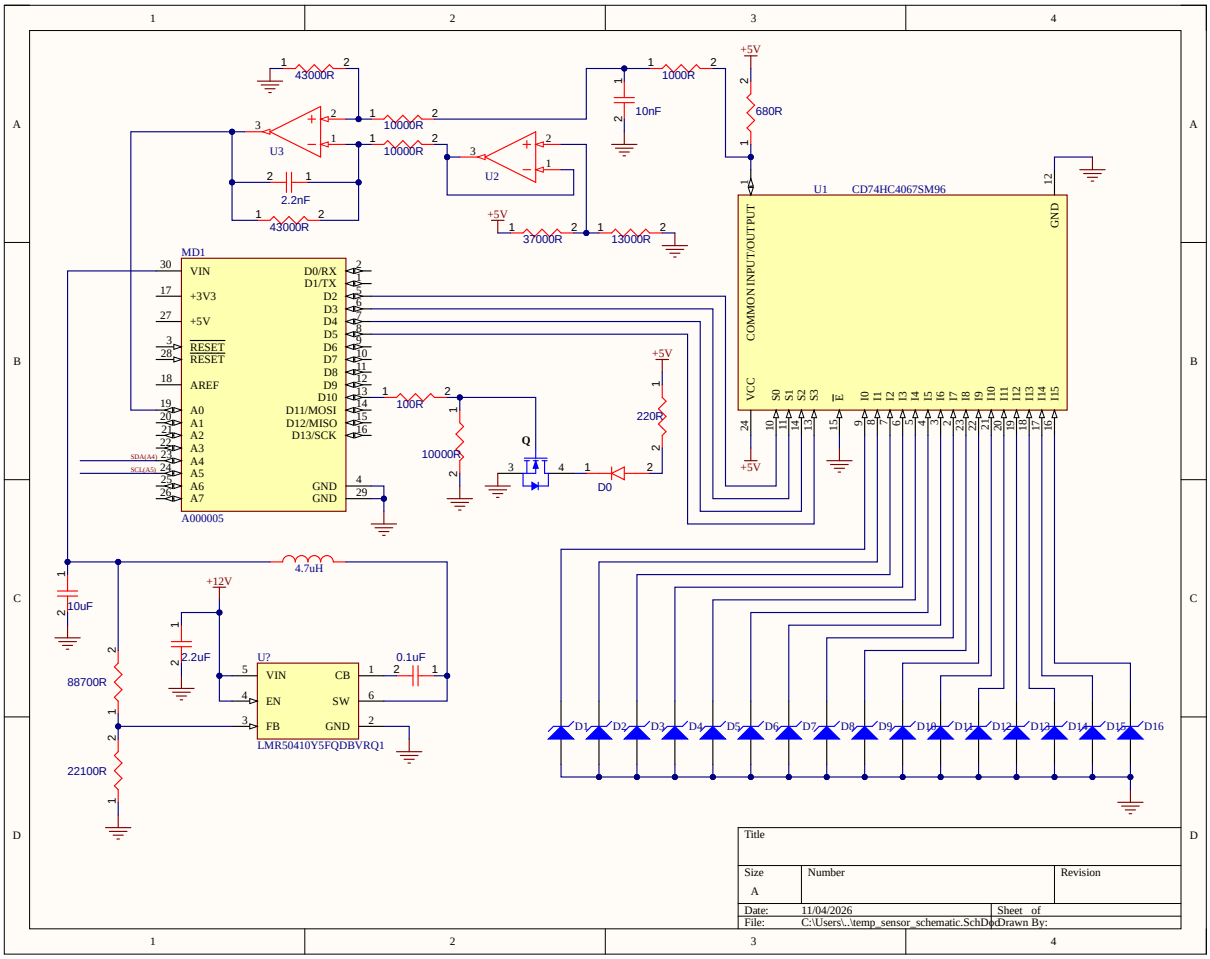


Figure 2: Schematic

4.1 Introduction to Schematic Design

Our schematic design contains 16 temperature sensors represented by zener diodes, a 16-channel multiplexer (CD74HC4067SM96), signal conditioning circuitry, an Arduino Nano, and a buck converter (LMR50410Y5FQDBVRQ1). This design allows the use of the Arduino Nano's onboard ADC to collect the voltage readings from each temperature sensor one by one with help from the 16-channel multiplexer. This is achieved by connecting each channel of the multiplexer to a temperature sensor reading. These channels are then controlled by the firmware in the Arduino Nano's D2-D5 pins, where each chosen output of the multiplexer is first conditioned, then sent to the A0 pin of the Arduino Nano, where the voltage is gathered and tested through the firmware. The MOSFET-controlled LED connected to the D10 pin then shines when the voltage reading interprets a temperature reading higher than 60 C.

4.2 RC Filter

- RC filter's main intention is to smooth out the sharp changes from the MUX switching between channels.
- 1 kΩ resistor will act as a current limiter. The resistor value cannot be too high, or it will affect the difference amplifier's 10 kΩ resistance value and mess with its overall gain.
- 10 nF capacitor is chosen due to time constant (τ).

Cutoff Frequency:

$$f_c = \frac{1}{2\pi RC} \approx 15.9\,\text{kHz}$$

4.3 Difference Amplifier

Temperature Sensor Voltage-Temperature Outputs:

Temperature:	$-40°\text{C}$	$\longrightarrow 60°\text{C} \longrightarrow$	$120°\text{C}$
Output Voltage:	$2.44\,\text{V}$	$\longrightarrow 1.81\,\text{V} \longrightarrow$	$1.30\,\text{V}$
	(min temp)	(target temp)	(max temp)

Difference Amplifier Circuit Analysis:
Intended to create a $0\,\text{V} - 5\,\text{V}$ output range from the temperature sensor readings ($1.30\,\text{V} - 2.44\,\text{V}$).
Transfer Function:

$$V_O = \frac{R_2}{R_1}(V_{\text{MUX}} - V_{\text{REF}}) \quad \left(\text{where } \frac{R_2}{R_1} \text{ is gain } A\right)$$

$$5 = A(2.44 - 1.3) \implies A = \frac{5\,\text{V}}{1.14\,\text{V}} \approx 4.386$$

Using standard resistor values:

$$R_1 = 10\,\text{k}\Omega \quad \text{and} \quad R_2 = 43\,\text{k}\Omega$$

$$\frac{43\,\text{k}\Omega}{10\,\text{k}\Omega} = 4.3 \implies A = 4.3 \quad \text{(ideal gain is truncated down to avoid clipping above 5 V)}$$

4.4 Voltage Divider & Buffer Op-Amp

Voltage Divider Calculation:
We want $V_{\text{REF}} = 1.3\,\text{V}$:

$$5\left(\frac{R_2}{R_1 + R_2}\right) = 1.3\,\text{V}$$

Using standard resistor values:

$$R_2 = 13\,\text{k}\Omega \quad \text{and} \quad R_1 = 37\,\text{k}\Omega$$

Buffer Op-Amp Analysis:

- Buffer op-amp will prevent **voltage loading**.
- Internal Thevenin resistance from R_1 & R_2 of the voltage divider will affect the R_1/R_2 ratio of the difference amplifier.
- Buffering will create a clone 1.3 V (provided by 5 V) that is clean like a 1.3 V battery.

4.5 Active Filtering ($2.2\,\text{nF}$ Capacitor)

$$gain(f) = \frac{R_f}{R_{\text{in}}}$$

$$Z_F = \frac{R_f}{1 + j2\pi f R_f C_F} \quad \text{with } C_F = 2.2\,\text{nF}$$

$$f_0 = \frac{1}{2\pi R_f C_F} \approx 1.682\,\text{kHz}$$

$$Z_f = 0\,\Omega \quad @ \quad f = 0\,\text{Hz}$$

$$Z_f = 43\,\text{k}\Omega \quad @ \quad f \approx 1.68\,\text{kHz}$$

- This means higher frequencies will short-circuit R_f, which will make the gain equal zero.
- This allows higher frequencies to be eliminated.

4.6 Settling Time Analysis

Settling time exists due to the time it takes for the filter's capacitors to charge/discharge every time the multiplexer switches channels.
Capacitor follows an exponential charging curve:

$$V(t) = V_S\left(1 - e^{-t/\tau}\right) \quad (\tau \text{ is time constant})$$

For an accurate reading from the 10-bit ADC on the Arduino, we need to consider the ADC resolution:

$$\frac{1}{2^{10}} = 0.000977 = 0.0977\%$$

To prevent sensor voltage reading overlap (crosstalk), the filter has to settle within 0.0977% of its final value. We need to find how many time constants are needed to reduce the error below $\frac{1}{2^{10}}$:

$$e^{-t/\tau} = \frac{1}{2^{10}} \implies \ln\left(e^{t/\tau}\right) = \ln\left(2^{10}\right)$$

$$\frac{t}{\tau} = 10\ln(2) \approx 6.93 \implies t \approx 6.93\tau$$

To prevent errors, round up: $t = 7\tau$.

Hardware Settling Breakdown

- **RC Filter:**
$$7\tau_1 = 7(1\,\text{k}\Omega \times 10\,\text{nF}) = 70\,\mu\text{s}$$
- **Active Feedback:**
$$7\tau_2 = 7(43\,\text{k}\Omega \times 2.2\,\text{nF}) = 662.2\,\mu\text{s}$$
- **Total Hardware Settling Time:**
$$70\,\mu\text{s} + 662.2\,\mu\text{s} = \underline{732.2\,\mu\text{s}}$$

4.7 Verifying 1000Hz Sampling Rate

- 1000 Hz sampling rate means it takes 1000 samples per second across all channels (1 ms per channel).
- Selecting MUX switch takes 1 μs (based off datasheet).
- Executing ADC conversion takes 112 μs (ranges from 100 μs − 125 μs).
- With our total hardware settling time being 732.2 μs...

$$t_{\text{tot}} = 1\,\mu\text{s} + 112\,\mu\text{s} + 732.2\,\mu\text{s} = 845.2\,\mu\text{s} = 0.8452\,\text{ms} < 1\,\text{ms}$$

With a headroom of 154.8 μs, it is safe to say it will achieve 1000 Hz sampling rate.

Note that this does not include delays included in our firmware design. If we include the added delay to account for the capacitor settling delay (20μs) we still will achieve the desired sampling rate.

4.8 Buck Converter

To verify that our buck converter (LMR50410Y5FQDBVRQ1) will work with our design, we need to validate the overall output current, which, according to the provided datasheet, needs to be less than 1A.

Total Output Currents

- **MOSFET LED:**
$$\frac{5V - 2V}{220\Omega} = 13.6mA$$
(2V is the max forward voltage drop from a Red LED)
- **Temperature Sensors:**
$$\frac{5V - 0.7V}{1000\Omega + 680\Omega} = 2.56mA$$
- **x2 LM358 Op-Amps**
$$2 \times 150\mu A = 0.3mA$$
- **Multiplexer:** 0.008mA (quiescent device current)
- **Arduino Nano:** 50mA (CPU consumption)
- **Total:**
$$13mA + 2.56mA + 0.3mA + 0.008mA + 50mA = 65.868mA < 1A$$

This is well under the limit of 1A

4.9 Schematic Changes in Implementation

- In actual implementation, we utilize an 8-channel multiplexer (CD4051BE). This change would reduce it to only 2 select line pins (S0-S1) and 8 data input pins (I0-I7), which is reflected in our firmware design structure.
- The buck converter is also not included in our implementation; rather, the Arduino Nano is powered through a laptop USB-C connection.

5 Firmware Design

5.1 Introduction and Firmware Overview

The software is based on our planned wiring implementation and serves four main functions per the project instructions: reading each sensor's analog signal, converting it to a voltage, determining the highest, lowest, and average temperature among the sensors, via linear interpolation, and reporting that value over I2C. While the original schematic includes a DC-DC converter and a 16:1 MUX, the converter is omitted here and an 8:1 MUX is used instead. This reduces the select pin count from four to three, while keeping the core firmware logic the same. An overview of our software from a high level is shown below in figure 3.

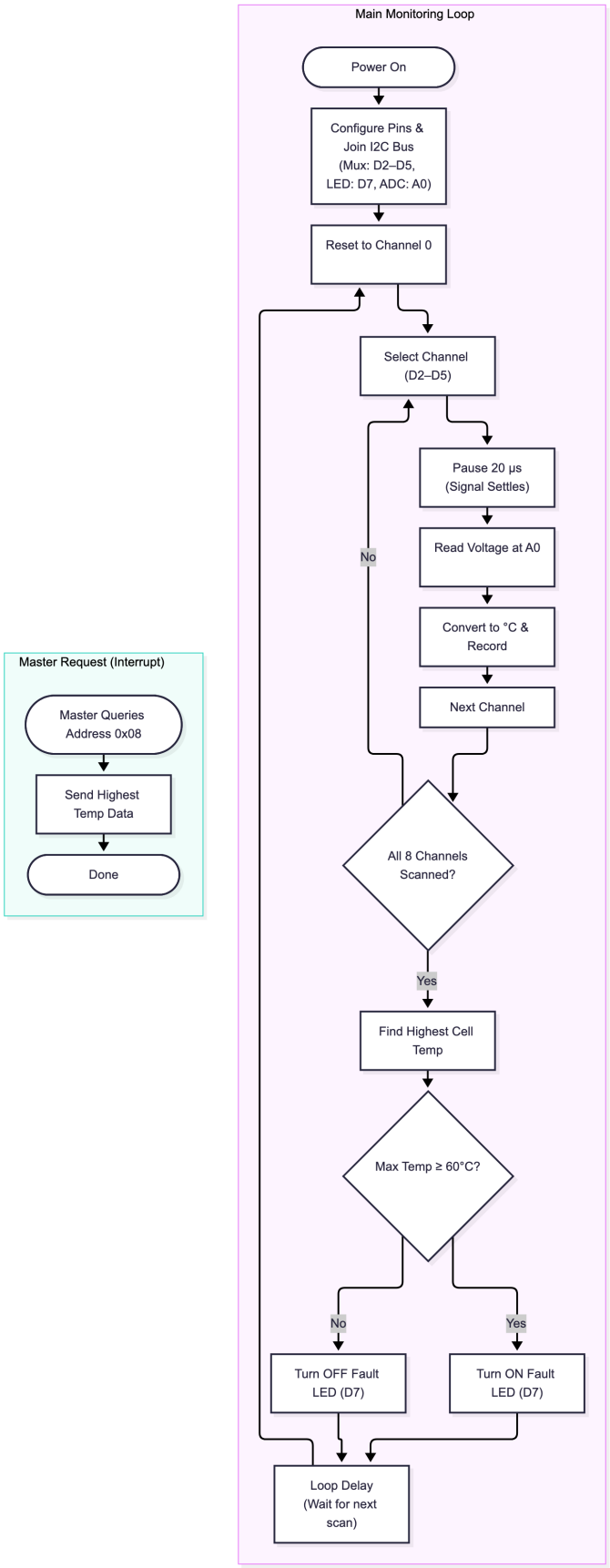


Figure 3: Software Flowchart

The core execution loop performs the following operations:

1. Iterates sequentially through channels 0 to 7 using the binary addresses from the select pins
2. Pauses for an explicit interval (20 μs) to discharge/charge front-end capacitance before sampling.
3. Samples the conditioned analog node via `analogRead(A0)` and converts the raw 10-bit count into an engineering voltage (0.0 V − 5.0 V).
4. Maps the measured voltage to temperature (°C) based on the Enepaq sensor's temperature and voltage mapping datasheet.
5. Evaluates cell temperatures against 60°C critical limit, asserting the fault LED on pin `D7` if overheating is detected.
6. Stores the maximum pack temperature to prepare for for I2C request

5.2 Pin Mapping and Peripheral Allocation

To match the schematic in Figure **??**, the hardware pin configuration are mapped out as followed:

Table 1: Microcontroller Pin Allocation

Constant Name	Nano Pin	Direction	Hardware Functional Description
`muxSIG`	A0	Input	Conditioned analog voltage from multiplexer
`muxS0`	D2	Output	Binary address select line: Bit 0 (2^0)
`muxS1`	D3	Output	Binary address select line: Bit 1 (2^1)
`muxS2`	D4	Output	Binary address select line: Bit 2 (2^2)
`muxS3`	D5	Output	Binary address select line: Bit 3 (2^3)
`faultLED`	D7	Output	Visual Indicator for overheating
`SDA`	A4	Bidirectional	I2C Data line
`SCL`	A5	Input	I2C Clock line

```
#include <Wire.h>
//Defiing all of our pins here

Signal Pin from multiplexer to the Arduino
const int muxSIG = A0;

Pins that will select what channel to read in binary
const int muxS0 = 2;
const int muxS1 = 3;
const int muxS2 = 4;


//Defining address for Arduino as Slave
const byte SLAVE_ADDRESS = 0x08;
```

Listing 1: Pin definitions

5.3 Overview of functions

5.3.1 Channel Addressing: `setMuxChannel`

The 8:1 Multiplexer needs a 3 bit binary value to select the channel. Instead of manually iterating through binary, we call this function to input the desired channel we want to read as a byte, and then it will convert it to the binary equivalent using Arduino's bit read function.

```
void setMuxChannel(byte channel)  {
  //Allows you to input an integer and then it converts it into binary for the signal pins to read and select the
     channel
    digitalWrite(muxS0, bitRead(channel, 0));
    digitalWrite(muxS1, bitRead(channel, 1));
    digitalWrite(muxS2, bitRead(channel, 2));
}
```

Listing 2: setMuxChannel Function

5.3.2 Signal Acquisition: `readChannelVoltage`

This function switches the channel, pauses for the signal to stabilize, takes a 10-bit analog sample, and converts the integer output to a floating point voltage using the voltagetoTemperature helper function.

```
float readChannelVoltage(byte channel) {
  setMuxChannel(channel);
  delayMicroseconds(20);

  int rawADC = analogRead(muxSIG);
  return voltagetoTemperature(rawADC);
}
```

Listing 3: readChannelVoltage Function

5.3.3 Mapping our voltage readings into temperature values: `voltagetoTemperature`

Since the ADC doesn't give us a voltage value that's scaled correctly, we have to rescale it from 0-5V using the following equation.

$$V_{\text{measured}} = \text{ADC} \times \left(\frac{5.0\,\text{V}}{1023.0} \right) \tag{1}$$

Once the value is converted, we use linear interpolation to map out the voltage from the lookup table given in the Enepaq data sheet, and convert the scaled value to the temperature.

Note: In our actual implementation we used 4.850 V as our ceiling voltage. The theoretical value of the voltage reading from the Adruino Nano is 5V, however, after testing the value with the multimeter, we find that it actually outputs 4.850 V

```
float voltagetoTemperature(int rawADC) {
//Converts rawADC value to actual voltage value between 0-5 volts
//It will be between 1.30V to 2.44V based on the diode
  float volt = (rawADC * 4.850f) / 1024.0f;
  //Linear interpolation using the two datasheet coordinates
  // (-40 C  @ 2.44V) and (120 C  @ 1.30V)
  const float V_cold = 2.44f;
  const float V_hot  = 1.30f;
  const float T_cold = -40.0f;
  const float T_hot  = 120.0f;

  float slope = (T_hot - T_cold) / (V_hot - V_cold); // ~ -140.3509 C/V.
  //For every voltage increase there is a -140 decrease in temperature. (negative slope)
  float temp  = T_cold + slope * (volt - V_cold);
  return temp;
}
```

Listing 4: Voltage to Temperature Conversion

Note: Our linear interpolation is very rough. We took data points from the highest and lowest temperature and found the slope creating a straight line curve to map the voltage. This is not reflective of the actual curve, and so our conversion values will be off. If given more time/AI assistance, we would create a struct and include all of the values from the look up table, then linear interpolate between neighboring points by looping through our array of temperature and voltages.

5.3.4 I2C Implementation: `requestData`

When the bus master addresses the slave node 0x08, the hardware triggers the request data function.

```
void requestData() {
  //send avg, highest, and lowest temperatures over I2C
  Wire.write((byte*)&avgTemp, sizeof(avgTemp));
  Wire.write((byte*)&maxTemp, sizeof(maxTemp));
  Wire.write((byte*)&minTemp, sizeof(minTemp));
}
```

Listing 5: I2C Slave Transmission Callback

5.4 Design Choices

5.4.1 Settling Delay Choice ($20\,\mu$s)

- **Context:** Inside the multiplexer, each switch adds about $70\,\Omega$ to $180\,\Omega$ of internal resistance This sits right before our input filter stage, which uses a $1\,\text{k}\Omega$ resistor and a $10\,\text{nF}$ capacitor before feeding the signal to the op-amp.
- **Alternative Considered:** Reading the ADC right after changing the channel address pins, with zero delay in between.
- **Decision & Rationale:** If we sample immediately, voltage left over on the capacitor from the previous channel will bleed into the new reading. The capacitor needs a moment to charge up to the new voltage level. We can find how long this takes using the circuit's RC time constant:

$$\tau = R_{\text{total}} \times C \approx (1000\,\Omega + 180\,\Omega) \times 10\,\text{nF} = 11.8\,\mu\text{s} \tag{2}$$

  Waiting $20\,\mu$s gives the signal about 1.7τ to settle down, ensuring our ADC reading reflects the true sensor voltage rather than residual charge from the channel before it. Across all 8 channels, this delay is negligible and we would not hinder our target of 1000hz sampling rate.

6 Bill of Materials

The following table itemizes all electronics required within our schematic. This BOM excludes some common found passive components (Through-hole resistors, capacitors, and LED diode), as well as the provided temperature sensors. All component prices and part numbers are sourced from Digikey.

Table 2: Project Bill of Materials (BOM)

Item #	Category	Vendor / SKU	Qty	Unit ($)	Total ($)	Status
A-01	Development Boards	Arduino / A000005	1	$25.70	$25.70	In Stock
A-02	Voltage Regulators	Texas Instruments / 296-LMR50410Y5FQDBVRQ1TR-ND	1	$2.32	$2.32	In Stock
A-03	Op Amps	Texas Instrument / 296-47593-2-ND	1	$0.33	$0.33	In Stock
A-04	Multiplexers	Texas Instruments / 296-2057-5-ND	1	$0.94	$0.94	In Stock
A-05	MOSFETs	Diotec Semiconductor / 4878-2N7000CT-ND	1	$0.29	$0.29	In Stock
				Subtotal:	$29.58	
				Est. Shipping:	$8.49	
				Tariff:	$0.26	
				Est. Total Cost:	**$38.33**	

7 Sources

References

[] Understanding Our Components

[1] Robotics Back-End, *Arduino Uno - Pins Overview*, YouTube, 2021. Available: https://youtu.be/bniUECtJkeU

[2] Electronoobs, *Arduino Multiplexor - MUX INPUTS & OUTPUTS - ANALOG and PWM*, YouTube, 2021. Available: https://youtu.be/Dco6jo9xgAo

[3] Dejan, *How I2C Communication Works?*, HowToMechatronics. Available: https://howtomechatronics.com/tutorials/arduino/how-i2c-communication-works-and-how-to-use-it-with-arduino/

[4] CircuitBread, *Intro to Op-Amps (Operational Amplifiers) | Basic Circuits*, YouTube, 2022. Available: https://youtu.be/fmRHDqcodS4

[5] Temperature Sensor Datasheet. Available: https://enepaq.com/wp-content/uploads/2025/02/Sony-Murata-VTC5A-Li-ion-Battery-Module-With-Temperature-Sensor-Datasheet-.pdf

[] Circuit Basics

[6] RimstarOrg, *How to Read a Schematic*, YouTube, 2014. Available: https://youtu.be/_HZ-EQ8Hc8E

[7] EEVblog, *Playlist on DC Fundamentals*, YouTube, 2017. Available: https://www.youtube.com/show/VLPLZJkSyc12jDY?sbp=KgtZZGJobm1BNE05Z0AB

[] Arduino IDE